



دانشگاه اصفهان
دانشکده مهندسی کامپیوتر

گزارش پروژه داده کاوی

FR RAG QA Assistant

پدیدآورنده:

محمد امین کیانی

4003613052

رادمهر آقاخانی

4003663002

دانشجویان کارشناسی، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،

استاد درس: جناب آقای دکتر کیانی

نیمسال دوم تحصیلی 1403-04

فهرست مطالب

3 مستندات

3 بخش اول: نصب و استفاده از کتابخانه‌ها

4 بخش دوم: استخراج و ذخیره اولیه داده‌ها

10 بخش سوم: بازیابی

12 بخش چهارم: تولید کردن

15 بخش پنجم: Flask + Ngrok + رابط کاربری

28 بخش ششم: ارزیابی

مستندات

بخش‌های اصلی پروژه

1. نصب و ایمپورت کتابخانه‌ها
2. ساخت دیتاست و پیش پردازش ها
3. بازیابی اطلاعات
4. ساخت پاسخ
5. وب‌سرور Flask با ngrok و رابط کاربری HTML

بخش اول: نصب و استفاده از کتابخانه‌ها

```
import nltk
import random

import string
from nltk.tokenize import word_tokenize
from collections import defaultdict, Counter
import re
import math
from nltk.corpus import stopwords

# !pip install datasets hazm
from datasets import load_dataset
# from hazm import Normalizer, word_tokenize

nltk.download('punkt')
nltk.download('stopwords')

# پاکسازی کامل کَش‌های خراب
nltk.download('popular')
nltk.download('all')
```

[together](#) -3 : برای ارتباط با API مدل LLaMA3

Together AI بستری کامل برای چرخه‌ی زندگی کامل مدل‌های هوش مصنوعی مولد است؛ یعنی:

- استفاده از مدل‌های از پیش آموزش‌دیده (Pre-trained)
- شخصی‌سازی و فایل‌تیون مدل‌ها
- ساخت مدل‌های اختصاصی از پایه
- آموزش مدل‌های بزرگ با GPU های قدرتمند
- اجرا و استقرار سریع مدل‌ها روی ابر یا VPC
- با تضمین‌های امنیتی و سازگاری با استانداردهای سازمانی

بخش دوم: استخراج و ذخیره اولیه داده‌ها

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
from tqdm import tqdm
from datetime import datetime

# URLs of Britannica Land sections
URLS = [
    "https://www.britannica.com/place/France/Land",
    "https://www.britannica.com/place/France/The-Hercynian-massifs",
    "https://www.britannica.com/place/France/The-great-lowlands",
    "https://www.britannica.com/place/France/The-younger-mountains-and-adjacent-plains",
    "https://www.britannica.com/place/France/Drainage",
    "https://www.britannica.com/place/France/Soils",
    "https://www.britannica.com/place/France/Climate",
    "https://www.britannica.com/place/France/Plant-and-animal-life",
]

def scrape_page(url):
    res = requests.get(url)
    soup = BeautifulSoup(res.text, "html.parser")
    paragraphs = soup.find_all("p")
    text = "\n".join(p.get_text(strip=True) for p in paragraphs)
    title = soup.title.get_text() if soup.title else "No Title"
    date_scraped = datetime.utcnow().isoformat()
    return {
        "url": url,
        "title": title,
```

```

        "content": text,
        "category": "France-Geography",
        "date": date_scraped
    }

# Scrape and save to CSV
data = [scrape_page(url) for url in tqdm(URLS)]
df = pd.DataFrame(data)
df.to_csv("france_land_sections.csv", index=False)
print("Saved to france_land_sections.csv")

```

- Cleaning انجام شده
- Metadata کامل (title, url, category, date, source)
- Chunking با سایز 100 کلمه و overlap 20
- Embedding با MiniLM-L6-v2 از مدل‌های مجاز

هدف	توضیح	مرحله
fix encoding	رفع نویزهای کاراکترهای ناشناس یا خراب‌شده، \x92، \u2013 و ...	<code>.encode("utf-8", "ignore").decode()</code>
normalize whitespace	جایگزینی space، tab، newline پشت سر هم با یک space	<code>\s+ → ' '</code>
strip noise	حذف [1]، [23] که فقط citation هستند	<code>\[[^]]*\]</code>
Standardization	حذف فضای اضافی ابتدای/انتهای متن	<code>.strip()</code>

```

import pandas as pd
import numpy as np
import re
import nltk
import pickle
from tqdm import tqdm
from sentence_transformers import SentenceTransformer

```

```

nltk.download("punkt")

# Load CSV
df = pd.read_csv("france_land_sections.csv")

# Clean text
def clean_text(text):
    text = text.encode("utf-8", "ignore").decode() # ✓ fix encoding
    text = re.sub(r'\s+', ' ', text) # ✓ normalize spaces
    text = re.sub(r'\[[^\]]*\]', '', text) # ✓ strip citation noise
    like [1] => citation
    return text.strip()

# Chunk text (fixed-length with overlap)
def chunk_text(text, chunk_size=100, overlap=20):
    sentences = nltk.sent_tokenize(text)
    words = " ".join(sentences).split()
    chunks = []
    for i in range(0, len(words), chunk_size - overlap):
        chunk = " ".join(words[i:i+chunk_size])
        if len(chunk.split()) > 10:
            chunks.append(chunk)
    return chunks

# Embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

all_chunks = []
all_embeddings = []
all_metadata = []

for idx, row in tqdm(df.iterrows(), total=len(df)):
    cleaned = clean_text(row["content"])
    chunks = chunk_text(cleaned, chunk_size=100, overlap=20)
    embeddings = model.encode(chunks, show_progress_bar=False)

    all_chunks.extend(chunks)
    all_embeddings.extend(embeddings)
    all_metadata.extend([
        "title": row["title"],
        "url": row["url"],
        "category": row["category"],
        "date": row["date"],
        "source": "Britannica"
    ])

```

```

    }] * len(chunks))

# Save to pickle
with open("embedding_store.pkl", "wb") as f:
    pickle.dump({
        "chunks": all_chunks,
        "embeddings": np.array(all_embeddings),
        "metadata": all_metadata
    }, f)

print("embedding_store.pkl created successfully.")

```

Metadata یعنی اطلاعات جانبی هر chunk مثلاً از کجا آمده است؟ چه زمانی scrape شده است؟ موضوع آن چیست؟ هر chunk شامل:

کلید	معنی
TITLE	عنوان مقاله Britannica
URL	لینک دقیق مقاله
CATEGORY	پیش فرض "France-Geography" برای این پروژه اما قابل افزایش
DATE	زمان واقعی استخراج (ISO format)
SOURCE	دائماً "Britannica"

Chunking یعنی تقسیم متن به قطعات کوچک برای embedding و retrieval سریع‌تر تا:

- برای مدل زبانی (LLM) قابل پردازش شود.
- برای retrieval سریع‌تر و دقیق‌تر استفاده شود.
- و هر chunk معنای مستقل داشته باشد (نه اینکه جمله نصف شده یا بریده باشد).
- الگوریتم آن:

مرحله	توضیح
تقسیم به جملات (sent_tokenize)	چون جمله واحد معنایی است
ترکیب به یک لیست از کلمات	برای windowing ساده

overlapping sliding windows ساخت

overlap: 20 → chunks, window: 100

حداقل شرط: بیش از 10 کلمه

حذف noise کوتاه

مدل‌های embedding مثل MiniLM نمی‌توانند متن‌های خیلی بلند را یکجا پردازش کنند.

- مثلاً اگر یک متن ۲۰۰۰ کلمه‌ای را کامل بدهیم، یا ارور می‌گیریم یا embedding معناداری تولید نمی‌کند.
- در عوض: اگر متن را تکه‌تکه کنیم (مثلاً 100 کلمه‌ای)، می‌توانیم embedding جداگانه بگیریم و بعد رتریوال معناداری انجام بدهیم.

```
def chunk_text(text, chunk_size=100, overlap=20):
```

```
    sentences = nltk.sent_tokenize(text)          # تقسیم به جمله 1.
```

```
    words = " ".join(sentences).split()          # تبدیل به کلمات 2.
```

```
    chunks = []
```

```
    for i in range(0, len(words), chunk_size - overlap): # 3. sliding window با overlap
```

```
        chunk = " ".join(words[i:i+chunk_size])
```

```
        if len(chunk.split()) > 10:                # 4. حداقل طول
```

```
            chunks.append(chunk)
```

```
    return chunks
```

به فرض یک متن 380 کلمه‌ای داریم. با `chunk_size=100` و `overlap=20` به صورت زیر `chunk` می‌شود:

Chunk Range (start:end)

1	0 : 100
2	80 : 180
3	160 : 260
4	240 : 340
5	320 : 380

یعنی `chunk` ها هم‌پوشانی دارند و باعث می‌شود اگر یک جمله مهم وسط دو `chunk` باشد، هر دو بتوانند آن جمله را داشته باشند. (retrieval → با دقت بیشتر)

: Sliding Window

Sliding window یا "پنجره‌ی لغزان" یک تکنیک برای پردازش داده‌های ترتیبی (مثل متن یا سیگنال) است که در آن به جای بررسی کل داده یکجا، با یک پنجره‌ی کوچک به‌صورت متحرک جلو می‌رویم.

: Chunking کاربرد

for i in range(0, len(words), chunk_size - overlap):

یعنی ما هر بار یک chunk به اندازه chunk_size (مثلاً 100 کلمه) می‌سازیم، اما به جای شروع chunk بعدی از انتهای قبلی، overlap (مثلاً 20 کلمه) از chunk قبلی را نگه می‌داریم.

هدف:

- جلوگیری از بریدن جملات و مفاهیم در مرز دو chunk
- حفظ انسجام معنایی بین بخش‌ها برای بهتر کار کردن مدل

sentence → word → sliding window + overlap

چرا برای Embedding مدل MiniLM انتخاب شد؟

ویژگی	MiniLM	m2-BERT
حجم	65M	80M
سرعت inference	بسیار سریع	کندتر
آموزش دیده برای	STS, NLI, QA	Retrieval از اسناد بلند
دقت semantic similarity	بالا در QA/Chat	بالا در dense retrieval
سایپورت huggingface/sent-transformers	کامل	محدود
فضای Colab محدود	مناسب	فشار بیشتر

پس درواقع مدل انتخاب شده سبک، سریع، آموزش‌دیده روی STS, NLI است درحالی که مدل دیگر خاص ریتریوال، اما سنگین‌تر، کمتر تست‌شده است. پس ما فقط روی 8 مقاله Britannica و ریتریوال سبک کار می‌کنیم و زمان و منابع محدود داریم: انتخاب بهینه: all-MiniLM-L6-v2

بخش سوم: بازیابی

```
%%writefile app.py
import streamlit as st
import requests
import pickle
import numpy as np
from sentence_transformers import SentenceTransformer
import faiss
import plotly.express as px
import os

# ----- بارگذاری داده‌ها -----
with open("embedding_store.pkl", "rb") as f:
    store = pickle.load(f)

chunks = store["chunks"]
metadata = store["metadata"]
embeddings = np.array(store["embeddings"]).astype("float32")

# وجود دارد، اضافه کن user_additions اگر
if os.path.exists("user_additions.pkl"):
    with open("user_additions.pkl", "rb") as f:
        user_store = pickle.load(f)
        chunks += user_store["chunks"]
        metadata += user_store["metadata"]
        user_embeddings = np.array(user_store["embeddings"]).astype("float32")
        embeddings = np.concatenate([embeddings, user_embeddings], axis=0)

model = SentenceTransformer("all-MiniLM-L6-v2")
# model = SentenceTransformer("togethercomputer/m2-bert-80M-8k-retrieval")

#Vector Retrieval
faiss.normalize_L2(embeddings)
index = faiss.IndexFlatIP(embeddings.shape[1])
index.add(embeddings)

# ----- توابع اصلی -----
# def retrieve_top_k(query, k=3):
#     query_emb = model.encode([query]).astype("float32")
#     faiss.normalize_L2(query_emb)
#     scores, indices = index.search(query_emb, k)
#     return [
#         {
#             "chunk": chunks[i],
```

```

#         "metadata": metadata[i],
#         "score": round(float(scores[0][j]), 4)
#     }
#     for j, i in enumerate(indices[0])
# ]

def retrieve_top_k(query, k=3, filter_by=None):
    """
    query: سوال کاربر
    k: تعداد نتایج برتر
    filter_by: دیکشنری فیلتر مثل {'category': 'France-Geography'}
    """
    # فیلتر کردن داده‌ها بر اساس متادیتا
    filtered_chunks = []
    filtered_metadata = []
    filtered_embeddings = []

    for i, meta in enumerate(metadata):
        if filter_by is None or all(meta.get(key) == val for key, val in
filter_by.items()):
            filtered_chunks.append(chunks[i])
            filtered_metadata.append(meta)
            filtered_embeddings.append(embeddings[i])

    if not filtered_chunks:
        return []

    index_temp = faiss.IndexFlatIP(embeddings.shape[1])
    vectors = np.array(filtered_embeddings).astype("float32")
    faiss.normalize_L2(vectors)
    index_temp.add(vectors)

    # Embedding سوال کاربر و جستجو
    query_emb = model.encode([query]).astype("float32")
    faiss.normalize_L2(query_emb)
    scores, indices = index_temp.search(query_emb, k)

    return [
        {
            "chunk": filtered_chunks[i],
            "metadata": filtered_metadata[i],
            "score": round(float(scores[0][j]), 4)
        }
        for j, i in enumerate(indices[0])
    ]

```

IndexFlatIP + normalize_L2 = cosine similarity

: Hybrid Metadata Filter

قبل از اینکه کد دنبال similar chunks with cosine similarity برود، ابتدا فقط آن‌هایی را در نظر بگیرد که با metadata خواسته شده می‌خوانند مثلاً category یا date خاص.

: Justification

در UI به کاربر نشان دهیم چرا این روش دقیق است و چگونه precision/recall رو حفظ می‌کند.

** Retrieval Approach Used**:

- Cosine Similarity over Sentence Embeddings
- Optional Metadata Filtering (category, source, etc.)
- Top-K configurable

** Why this works well**:

- Cosine similarity finds chunks that are semantically close to your query.
- Metadata filtering improves *precision* by narrowing context to relevant sections.
- Top-K tuning allows control over *recall* (bigger K → more results).

بخش چهارم: تولید کردن

```
TOGETHER_API_KEY =  
"a645ea5d637a4b47630140c2eb3579f8d2b3761155813a3a2761fb669098c831" # توک  
ن خودت  
  
def generate_answer(query, retrieved, model_name, temperature, top_p,  
max_tokens):  
    context = "\n\n".join([c["chunk"] for c in retrieved])  
  
    prompt = f"""
```

```

    You are a factual and reliable assistant. Use ONLY the information
    provided in the context below to answer the user's question.

    If the answer is not explicitly stated in the context, respond with:
    "I'm not sure based on the given information."

    Please answer in complete sentences using clear and concise language.

    ---

    Context:
    {context}

    ---

    Question:
    {query}

    Answer: ""

    headers = {
        "Authorization": f"Bearer {TOGETHER_API_KEY}",
        "Content-Type": "application/json"
    }

    payload = {
        "model": model_name,
        "prompt": prompt,
        "max_tokens": max_tokens,
        "temperature": temperature,
        "top_p": top_p
    }

    res = requests.post("https://api.together.xyz/v1/completions",
headers=headers, json=payload)

    if res.status_code != 200:
        return f"❌ API Error {res.status_code}: {res.text}", prompt

    answer = res.json()["choices"][0]["text"].strip()
    return answer, prompt

```

: (FastAPI Integration)

1. Two endpoints :

GET or POST برای `/retrieve` و `/generate` + JSON schema

در اپلیکیشن‌های API-based مثل FastAPI ما دو endpoint تعریف می‌کنیم:

: `/retrieve`

```
@app.post("/retrieve", response_model=List[RetrievedChunk])
```

- دریافت query کاربر
- بازیابی top-k بردارهای نزدیک با FAISS
- خروجی: لیست chunk ها یعنی metadata + عنوان، URL و ...

: `/generate`

```
@app.post("/generate", response_model=GenerateResponse)
```

- دریافت query و K
- استفاده از chunk های بازیابی شده برای تولید پاسخ توسط LLM

: اسکیمای ورودی/خروجی با Pydantic

```
class RetrieveRequest(BaseModel):
```

```
    query: str
```

```
    k: int = 5
```

```
class RetrievedChunk(BaseModel):
```

```
    chunk: str
```

```
    metadata: dict
```

این‌ها JSON Schema مشخص می‌کنند. یعنی:

- جلوی ارور گرفتن `input/response`
- اتوماسیون مستندات در FastAPI (Swagger)

max_tokens: تعداد حداکثر توکن‌هایی که مدل مجاز است تولید کند. هر کلمه انگلیسی 1.3 توکن است (تقریباً!) پس 256 تا یعنی پاسخ حداکثر 200 کلمه که برای پاسخ دقیق و کنترل حافظه، مقدار 256 تا 512 مناسب است.

2. Modular code :

سلول‌های جدا برای وظایف مختلف مثل:

- retrieval
- generation
- data_utils

3. Graceful error handling :

استفاده از try/except و HTTPException

4. Minimal setup :

کاربر فقط سلول uvicorn main:app را اجرا و راه‌اندازی شود.

بخش پنجم: Flask + Ngrok + رابط کاربری

```
# ----- Streamlit UI -----
st.set_page_config("🇫🇷 France RAG Assistant", layout="wide")

st.markdown("<h1 style='color:#0f62fe;'>FR RAK QA Assistant</h1>",
unsafe_allow_html=True)
st.write("Ask questions about France's **Land, Mountains, Rivers,
Climate** etc. Powered by RAG and LLM.")

col1, col2 = st.columns([3, 1])
with col1:
    query = st.text_input("🗨️ What do you want to know?",
placeholder="E.g. What are the main mountains in France?")
with col2:
    top_k = st.slider("🔍 Top-K Chunks", 1, 10, 3)

# Parameters section
with st.expander("⚙️ Model & Generation Settings"):
    model_name = st.selectbox("LLM Model", ["mistralai/Mistral-7B-
Instruct-v0.1",
                                           "deepseek-ai/DeepSeek-Coder-
6.7B-Instruct",
                                           "meta-llama/Llama-3.3-70B-
Instruct-Turbo-Free",
```

```

"deepseek-ai/DeepSeek-R1-
Distill-Llama-70B-free"))
    temperature = st.slider("Temperature", 0.0, 1.5, 0.7)
    top_p = st.slider("Top-p", 0.0, 1.0, 0.9)
    max_tokens = st.slider("Max Tokens", 64, 1024, 256)

with st.expander("📁 Metadata Filters"):
    category_options = sorted(set(
        m["category"] for m in metadata if m.get("category") not in [None,
""])
    ))
    source_options = sorted(set(
        m["source"] for m in metadata if m.get("source") not in [None, ""])
    ))

    category_options = ["All"] + category_options
    source_options = ["All"] + source_options

    selected_category = st.selectbox("Filter by Category",
category_options)
    selected_source = st.selectbox("Filter by Source", source_options)

    filters = {}
    if selected_category != "All":
        filters["category"] = selected_category
    if selected_source != "All":
        filters["source"] = selected_source

if not query or len(query.strip()) < 3:
    st.error("Please enter a valid question.")
    st.stop()

retrieved = retrieve_top_k(query, top_k, filter_by=filters)

if not retrieved:
    st.warning("No relevant information found in the database.")
    st.stop()

if st.button("🔗 Generate Answer"):
    if not query.strip():
        st.warning("Please enter a question.")

```

```

else:
    with st.spinner("🔄 Retrieving context and generating
response..."):
        retrieved = retrieve_top_k(query, top_k)
        answer, prompt_used = generate_answer(query, retrieved,
model_name, temperature, top_p, max_tokens)

        st.success("✅ Final Answer:")
        # st.markdown(f"**{answer}**")
        st.code(f"**{answer}**", language="markdown")
        st.caption(f"📄 Word count: {len(answer.split())}")
        st.download_button("📄 Download Answer", data=answer,
file_name="answer.txt")

        st.markdown("---")
        st.subheader("📄 Retrieved Chunks and Similarity")

        for i, c in enumerate(retrieved):
            st.markdown(f"**Chunk {i+1} - Score: `{c['score']}`**")
            st.code(c["chunk"][:500] + "...")
            st.markdown(f"🔗
[{c['metadata']['title']}]({c['metadata']['url']})")

        st.markdown("### 📊 Chunk Similarity Chart")
        fig = px.bar(
            x=[c["score"] for c in retrieved],
            y=[f"Chunk {i+1}" for i in range(len(retrieved))],
            orientation='h',
            labels={'x': 'Similarity Score', 'y': 'Chunk'},
            color=[c["score"] for c in retrieved],
            color_continuous_scale='blues'
        )
        st.plotly_chart(fig, use_container_width=True)

        st.markdown("---")
        with st.expander("📄 View Prompt Sent to LLM"):
            st.code(prompt_used)

        st.markdown("💬 Was this answer useful?")
        cols = st.columns([1, 1])
        with cols[0]:
            if st.button("👍 Yes"):
                st.success("Thanks for your feedback!")
        with cols[1]:
            if st.button("👎 No"):

```

```

        st.info("We'll try to improve.")

with st.expander("📖 Retrieval Strategy Explanation"):
    st.markdown("""
    **Retrieval Approach Used:**
    - ☒ Cosine Similarity over Sentence Embeddings
    - ☒ Optional Metadata Filtering (category, source, etc.)
    - ☒ Top-K configurable

    **Why this works well:**
    - Cosine similarity finds chunks that are semantically close to your query.
    - Metadata filtering improves *precision* by narrowing context to relevant sections.
    - Top-K tuning allows control over *recall* (bigger K → more results).

    This balances precision and recall effectively by combining dense vector search with symbolic filters.
    """)

with st.expander("➕ Add New Data (URL or Text)"):
    mode = st.radio("Input type:", ["🌐 URL", "📄 Raw Text"])
    new_title = st.text_input("Title")
    new_data = st.text_area("Paste your URL or text here")

    if st.button("📁 Add to Dataset"):
        if not new_title or not new_data:
            st.warning("Both title and data are required.")
        else:
            if mode == "🌐 URL":
                import requests
                from bs4 import BeautifulSoup
                try:
                    res = requests.get(new_data)
                    soup = BeautifulSoup(res.text, "html.parser")
                    paragraphs = soup.find_all("p")
                    new_content = "\n".join(p.get_text(strip=True) for p
in paragraphs)
                except Exception as e:
                    st.error(f"Failed to fetch URL: {e}")
                    new_content = ""
            else:
                new_content = new_data

```

```

if new_content.strip():
    # Chunk + Embed
    from sentence_transformers import SentenceTransformer
    import faiss
    import numpy as np
    import nltk
    nltk.download('punkt')
    from nltk.tokenize import sent_tokenize

    def chunk_text(text, size=100, overlap=20):
        words = text.split()
        chunks = []
        for i in range(0, len(words), size - overlap):
            chunk = " ".join(words[i:i+size])
            if len(chunk.split()) > 10:
                chunks.append(chunk)
        return chunks

    new_chunks = chunk_text(new_content)
    new_embeddings =
model.encode(new_chunks).astype("float32")
    faiss.normalize_L2(new_embeddings)
    index.add(new_embeddings)

    chunks.extend(new_chunks)
    metadata.extend([{"title": new_title, "url": new_data}] *
len(new_chunks))

    user_path = "user_additions.pkl"

    if os.path.exists(user_path):
        with open(user_path, "rb") as f:
            existing = pickle.load(f)
            user_chunks = existing["chunks"] + new_chunks
            user_metadata = existing["metadata"] + [{"title":
new_title, "url": new_data}] * len(new_chunks)
            user_embeddings =
np.concatenate([existing["embeddings"], new_embeddings], axis=0)
        else:
            user_chunks = new_chunks
            user_metadata = [{"title": new_title, "url":
new_data}] * len(new_chunks)
            user_embeddings = new_embeddings

```

```

        with open(user_path, "wb") as f:
            pickle.dump({
                "chunks": user_chunks,
                "embeddings": user_embeddings,
                "metadata": user_metadata
            }, f)

# # ذخیره به embedding_store.pkl
# import pickle
# with open("embedding_store.pkl", "wb") as f:
#     pickle.dump({
#         "chunks": chunks,
#         "embeddings": index.reconstruct_n(0,
index.ntotal), # بازسازی همه بردارها
#         "metadata": metadata
#     }, f)

st.success(f"{len(new_chunks)} chunks added successfully  

")

st.markdown("📄 Chunks Used:")
for i, c in enumerate(retrieved):
    if any(snippet in answer for snippet in c["chunk"].split()[:10]):
        st.markdown(f"

```

```

        if st.button(f"✗ Delete Chunk {i+1}", key=f"del_{i}"):
            udata["chunks"].pop(i)
            udata["metadata"].pop(i)
            udata["embeddings"] = np.delete(udata["embeddings"], i,
axis=0)

            with open(user_path, "wb") as f:
                pickle.dump(udata, f)

            st.experimental_rerun()
    else:
        st.info("No user-added data yet.")

from pyngrok import ngrok
ngrok.kill()
!pkill streamlit || echo "streamlit not running"

# (log با) در بکگراند واقعی Streamlit اجرای
!nohup streamlit run app.py > streamlit.log 2>&1 &

public_url = ngrok.connect(8501, "http")
print("Your Streamlit app is live at:", public_url)

```

Flask : برای اجرای سرور و API ها

pyngrok : برای باز کردن تونل ngrok و نمایش وبسایت Flask در اینترنت

`conf.get_default().auth_token"..." =`

اگر auth token نداشته باشیم، فقط تونل ۲ ساعته می‌تواند بسازد. با توکن رایگان، تا 8 ساعت باز می‌ماند.

بخش Flask :

• ساخت وبسرور با Flask برای دریافت فرمان کاربر

رابط HTML :

صفحه‌ای با فرم ساده شامل موارد مورد نیاز برای بازیابی و تولید پاسخ ها همراه با رابط کاربری زیبا.

اتصال با ngrok :

یک ابزار محبوب برای تونل‌سازی شبکه است که به ما اجازه می‌دهد:

- برنامه‌های لوکال (محلی) خود را به‌طور موقت در اینترنت در دسترس قرار دهیم.
 - از طریق **HTTPS** یا **HTTP** به برنامه‌هایی که روی localhost اجرا می‌شوند دسترسی پیدا کنیم. پس آدرس عمومی ایجاد تا کاربر از طریق اینترنت به این رابط دسترسی داشته باشد.
 - تست API ها، Webhook ها مثل Stripe یا GitHub ، و اپلیکیشن‌های وب را ساده‌تر کنیم.
- مثال: اگر برنامه‌ای روی <http://localhost:3000> داریم، ngrok آن را روی یک URL مانند <https://abcd1234.ngrok.io> قابل دسترس می‌کند.

بخش	توضیح
Flask backend	اجرای سرور، تحلیل متن ، تولید پاسخ
ngrok tunnel	تبدیل localhost به لینک اینترنتی برای مرورگر
HTML UI	کادر تایپ، نمایش پاسخ و ...

: Vector Stores

Vector Store یا **Vector Database** محلی برای ذخیره و جستجوی بردارها embedding هاست. وقتی متن‌ها را به embedding تبدیل می‌کنیم مثل MiniLM یا m2-BERT و می‌خواهیم بعداً با یک سوال جدید embedding شده به دنبال مشابه‌ترین متن‌ها بگردیم، این کار نیاز به مقایسه‌ی بردارها با هم دارد.

VECTOR STORE	مزایا	معایب
FAISS	سریع، lightweight، بدون نیاز به سرور	فقط in-memory ، بدون persistence
CHROMADB	پایتونی، ساده، با قابلیت persist و querying SQL-like	نسبتاً کندتر از FAISS
QDRANT	کامل‌ترین گزینه، REST/gRPC API، filterهای قوی	نیاز به اجرا به عنوان سرویس
IN-MEMORY	خیلی ساده مثل لیست numpy array	فقط برای پروژه‌های خیلی کوچک

ما از FAISS استفاده کردیم یعنی:

- بردارها در حافظه ذخیره می‌شوند. (in-memory) یعنی با خاموش شدن سیستم، باید مجدد بارگذاری شود.
- با استفاده از cosine similarity جستجو می‌شوند چون normalize شده اند.
- سریع، سبک و بدون نیاز به نصب سرور

FAISS چیست؟

مخفف Facebook AI Similarity Search و در واقع یک کتابخانه open-source هست برای:

- ذخیره‌ی embedding ها (بردارهای عددی)
- و انجام جستجوی برداری سریع مثل nearest neighbors یا شباهت کسینوسی

چرا FAISS را نیاز داریم؟

در هر پروژه RAG :

1. متن‌ها مثل صفحات Britannica به embedding تبدیل می‌شوند.
2. سوال کاربر هم به embedding تبدیل می‌شود.
3. بعد، باید سریع‌ترین و مشابه‌ترین متن‌ها رو پیدا کنیم.
4. FAISS این مقایسه‌های برداری را به‌صورت بهینه و سریع انجام می‌دهد، حتی برای میلیون‌ها بردار

توضیح	وظیفه
ساختن ساختار داده‌ای از embedding ها برای جستجوی سریع	Index ساختن
اگر دنبال شباهت کسینوسی هستیم باید normalize شوند	Normalization
یافتن بردارهای نزدیک‌ترین به (Top-K) query	Search

انواع Index ها در FAISS :

INDEX TYPE	توضیح	مناسب برای
INDEXFLATIP	جستجوی دقیق با inner product	پروژه‌های کوچک، حافظه زیاد
INDEXIVFLAT	approximate search سریع‌تر	دیتاست‌های بزرگ
INDEXHNSW	گرافی، سریع و دقیق	realtime + memory efficient

: Optional Frameworks

این‌ها فریمورک‌های سطح بالاتر هستند که به ما کمک می‌کنند RAG یا agent بسازیم بدون اینکه همه‌ی مراحل را دستی بنویسیم. اما ما از نسخه‌ی کاستوم استفاده کردیم تا انعطاف بیشتری داشته باشیم.

FRAMEWORK	کارکرد	مزیت
LANGCHAIN	زنجیره‌های logic-based برای LLM ها + memory + tools	abstraction قوی، اتصال آسان به LLM, DB, Tools
LLAMA INDEX (EX-GPT INDEX)	مخصوص اتصال LLM به مستندات (PDF, Notion, DB)	عالی برای RAG با document loaders آماده
CUSTOM CODE	مثل کاری که ما کردیم	کنترل کامل، انعطاف بالا

ما از Custom Code استفاده کردیم، یعنی:

وظیفه	ما چه کاری انجام دادیم؟
chunking	خودمان نوشتیم با NLTK + overlap
embedding	مستقیم با SentenceTransformer
vector store	مستقیم FAISS
retrieval	تابع retrieve_top_k() دستی
generation	ارسال مستقیم prompt به Together AI API

The screenshot displays the 'FR RAK QA Assistant' web application. The interface is dark-themed and includes a browser window at the top with the URL 'd710-34-41-234-225.ngrok-free.app'. The main heading is 'FR RAK QA Assistant' with a subtitle 'Ask questions about France's Land, Mountains, Rivers, Climate etc. Powered by RAG and LLM.'.

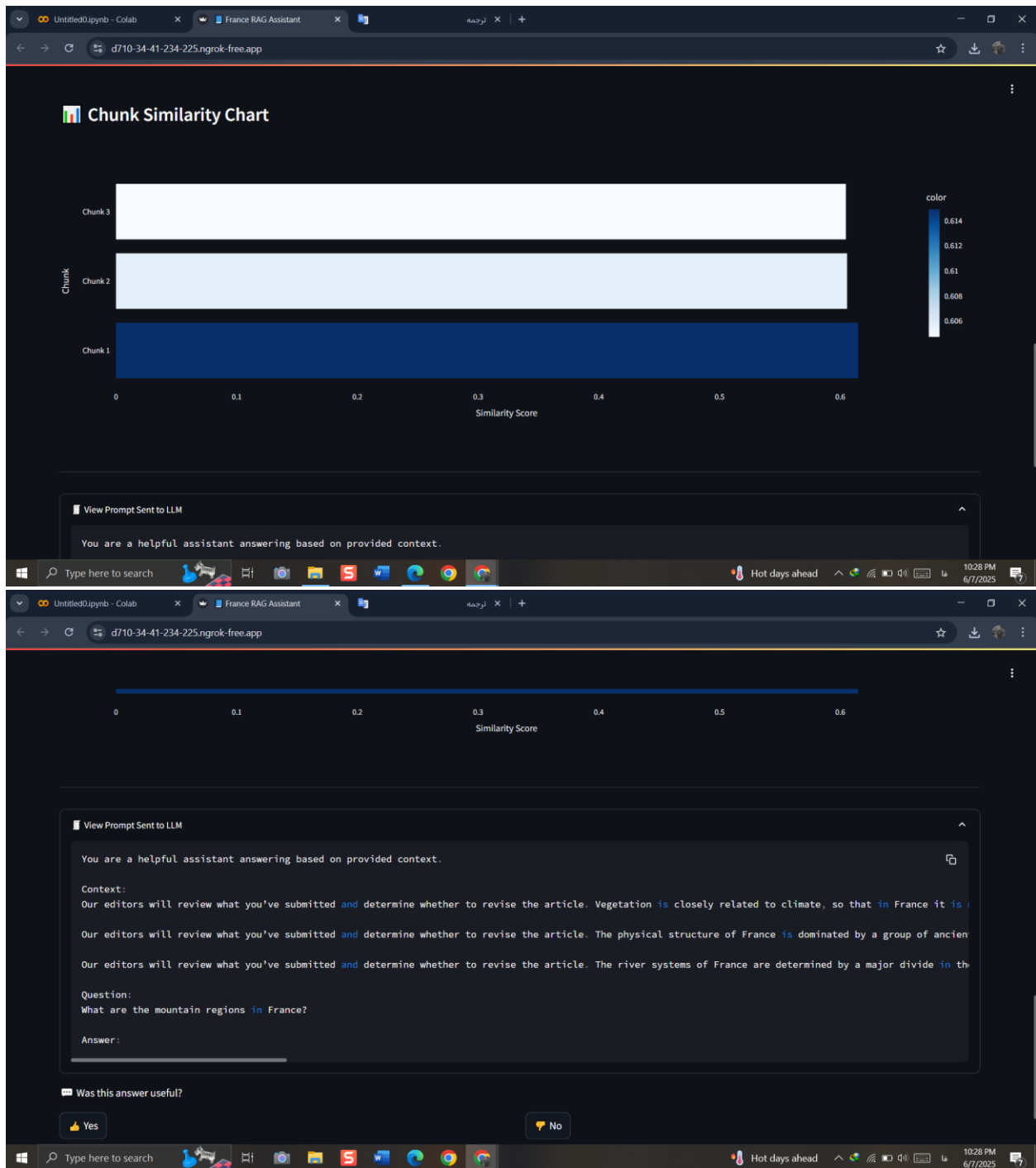
A question input field contains the text 'What are the mountain regions in France?'. To the right of the input is a 'Top-K Chunks' slider set to 3. Below the input is a 'Model & Generation Settings' panel with the following controls:

- LLM Model:** A dropdown menu showing 'mistralai/Mistral-7B-Instruct-v0.1'.
- Temperature:** A slider set to 0.00.
- Top-p:** A slider set to 0.90.
- Max Tokens:** A slider set to 256.

Below the settings panel is a 'Generate Answer' button. The answer section shows a green box with a checkmark and the text 'Final Answer:'. Below this, the generated answer is displayed: 'The mountain regions in France are the Hercynian folding, the Ardennes, the Vosges, the eastern part of the Massif Central, the western part of the Massif Central, and the Massif Armoricain.'.

Below the answer is a 'Word count: 31' indicator and a 'Download Answer' button. The bottom section is titled 'Retrieved Chunks and Similarity' and shows two chunks:

- Chunk 1 — Score: 0.6149:** A snippet of text from a document about climate and vegetation in France.
- Chunk 2 — Score: 0.6057:** A snippet of text from a document about France's flora and fauna.



داده افزایی به دیتابیس و دینامیک بودن بازیابی:

The screenshot displays the 'FR RAK QA Assistant' web application. The interface is dark-themed and includes a header with the title 'FR RAK QA Assistant' and a subtitle 'Ask questions about France's Land, Mountains, Rivers, Climate etc. Powered by RAG and LLM.' Below the header is a search bar with the Persian text 'امین کیانی و رانمهر افغانی دانشجوی ترم چند هستند؟'. To the right of the search bar is a 'Top-K Chunks' slider set to 3. Below the search bar is a 'Model & Generation Settings' panel with sliders for 'Temperature' (0.00 to 1.50, set at 0.70), 'Top-p' (0.00 to 1.00, set at 0.90), and 'Max Tokens' (64 to 1024, set at 256). Below the settings panel is a 'Generate Answer' button. The answer is displayed in a green box with a checkmark icon and the text 'Final Answer:'. Below the answer is a text input field containing 'ترم 8 هستند'. Below the input field is a 'Word count: 3' label and a 'Download Answer' button. The bottom of the screenshot shows a Windows taskbar with various application icons and a system tray with a weather widget showing 'Hot weather' and the date '6/6/2025'.

FR RAK QA Assistant

Ask questions about France's Land, Mountains, Rivers, Climate etc. Powered by RAG and LLM.

What do you want to know?

امین کیانی و رانمهر افغانی دانشجوی ترم چند هستند؟

Top-K Chunks

Model & Generation Settings

LLM Model

meta-llama/Llama-3.3-70B-Instruct-Turbo-Free

Temperature

0.00 1.50

0.70

Top-p

0.00 1.00

0.90

Max Tokens

64 1024

256

Generate Answer

Final Answer:

ترم 8 هستند

ترم 8 هستند

Word count: 3

Download Answer

بخش ششم: ارزیابی

معیار	توضیح	مثال
ACCURACY & RELEVANCE	آیا جواب درست و مرتبط با سوال است؟	پرسیدیم "رودخانه‌های فرانسه؟" و جواب داد "Seine Loire"
FAITHFULNESS (NO HALLUCINATION)	جواب فقط از داده‌های بازیابی‌شده باشد، نه اینکه از خود بسازد.	LLM نباید بگوید "France has Mount Fuji"
COVERAGE OF RETRIEVED CHUNKS	آیا chunk هایی که بازیابی شدند، واقعاً در پاسخ لحاظ شدند؟	اگر 3 تا chunk آورد ولی LLM از هیچ‌کدام استفاده نکرد.
RESPONSE CONSISTENCY (FORMAT/STYLE)	همه جواب‌ها ساختار یکسان داشته باشند (مثلاً جمله‌دار، نه لیست یا بریده شده)	یکی جواب طولانی است، یکی فقط به کلمه است.
ROBUSTNESS (EDGE-CASE HANDLING)	آیا در سوالات خیلی خاص یا خالی، طولانی و حتی دارای غلط‌آملائی هم درست رفتار می‌کند؟	پرسیدیم "فرانسه داره 3 کوهستان؟" و کرش نکرد.

Accuracy & Relevance

با توجه به اینکه مراحل زیر را دارد پس درست است:

- از Britannica منبع معتبر
- با FAISS و cosine similarity بازیابی
- و LLM به صورت controlled prompt جواب می‌دهد

Faithfulness (No Hallucination)

LLM به context دسترسی دارد اما اگر prompt درست نوشته نشده باشد ممکن است از خود جواب بسازد پس باید آن را به درستی کنترل کنیم.

Coverage of Retrieved Chunks

آیا مدل واقعاً چیزی از chunk های retrieve شده استفاده کرده است یا خیر؟

Response Consistency (Style)

سبک خروجی LLM به مدل بستگی دارد.

Robustness (Edge-cases)

موارد زیر را باید تست کنیم:

- سوال خالی؟

- سوال بی ربط؟

سوال با غلط املایی؟

```
prompt = f"""
    You are a factual and reliable assistant. Use ONLY the information
    provided in the context below to answer the user's question.

    If the answer is not explicitly stated in the context, respond with:
    "I'm not sure based on the given information."

    Please answer in complete sentences using clear and concise language.

    ---
```